

AgentArk: A General-Purpose Environment Framework for Scalable Multimodal Agent Evaluation and Reinforcement Learning

Jiage Liang
Independent Researcher
agentark@qq.com

Abstract

Multimodal agents are moving from static perception toward closed-loop interaction: a model must observe an environment, choose actions, receive feedback, and improve over repeated attempts. This shift makes environment supply a central bottleneck. We present AgentArk, a general-purpose environment framework centered on loadable task modules and an LLM-native runtime shell. AgentArk separates a reusable environment runtime, task modules, a Python toolkit, and a Hub site for task pages, results, and reproducibility artifacts. Task modules define observations, allowed actions, scoring rules, and termination conditions, allowing the same runtime to host a broad class of multimodal tasks that can be expressed through visible or textual state, executable interaction, and verifiable feedback. AgentArk scales environment supply through a multi-agent task production workflow, where coordinator, designer, builder, and reviewer agents turn task ideas into verified reusable modules. The Python toolkit supports model evaluation, replay, parallel execution, and reinforcement learning interfaces. AgentArk is designed not as a fixed task set or a single-domain simulator, but as infrastructure for continuously growing multimodal training environments. Its runtime shell replaces fixed-shape action and observation assumptions with code-as-action programs, macro-level task APIs, contextual observations, and explicit error feedback. The public release is organized around 32 listed tasks, 31 public scoreboards, a runnable environment package, task modules, evaluation and RL tooling, replay traces, and public Hub results from recorded model-environment interactions.

Keywords: multimodal agents; reinforcement learning; environment scaling; code-as-action; game-engine environments; task generation

1 Introduction

Large language model agents have made rapid progress in software engineering, tool use, and long-horizon reasoning, but general multimodal intelligence requires models to act inside environments that respond and change. In such settings, the relevant ability is not merely to describe an image, but to make decisions from visual and textual observations and to learn from failures. ESI-Bench emphasizes the perception-action loop [1], while ARC-AGI-3 frames interactive rule discovery and in-context learning as core evaluation targets [2]. Together, these trends point to a need for interactive, evaluable, and trainable environments rather than static visual question-answering datasets.

Building such environments requires three properties at the same time. First, tasks must be verifiable, with clear scores and termination conditions. Second, runs should be reproducible enough to support fair evaluation, replay debugging, ablations, and group-based reinforcement learning. Third, the task supply must scale, because a fixed benchmark can be memorized, saturated, or overfit. World models show a promising route toward open-ended interactive worlds [3, 4], but a training environment also needs task goals, action interfaces, scoring rules, and reproducible experiment artifacts. AgentArk starts from verifiable tasks and connects visible scenes, actions, and scoring through an engineering-oriented framework.

Our design judgment is that coding agents can currently complement generative worlds by constructing the task logic, validators, and interfaces that make environments trainable. If an agent can implement scene logic, checking rules,

and interaction interfaces, it can also help generate multimodal training environments. This does not reject the world-model direction; rather, AgentArk offers a complementary route focused on verifiable tasks, replayable traces, and trainable interfaces.

This paper makes the following contributions:

- We introduce a scalable multimodal-agent environment framework that decouples the runtime, task modules, Python toolkit, and Hub site.
- We define an LLM-native runtime shell in which task instructions, multimodal observations, action schemas, scoring rules, and termination conditions jointly specify a task.
- We describe the multi-agent task production workflow that scales task supply through coordinator, designer, builder, reviewer, and bounded fix-review stages.
- We describe how AgentArk extends the Gym-style step loop with code-as-action programs, macro-level actions, context-learning observations, error feedback, and trace recording.
- We implement evaluation, replay, parallel execution, isolated runtime workers, pre-started environment pools, and service interfaces for reinforcement learning.
- We report a public task catalog with 32 listed tasks, 31 public scoreboards, and initial multi-model evaluation records and aggregate results, demonstrating AgentArk’s extensibility as an environment engine.

2 Related Work

World-model approaches aim to generate interactive worlds directly. Genie 3 points toward real-time explorable environments from text descriptions [3], and SIMA 2 explores multimodal agents that act, reason, and learn in 3D virtual worlds [4]. Viewed together, this line emphasizes increasingly open and dynamic worlds. AgentArk represents a complementary task-and-verification route: it prioritizes task goals, scoring, and replay, and can later benefit from 3D asset generation or world models as richer sources of visual and dynamic variation.

Interactive multimodal benchmarks are also developing quickly. GameWorld provides verifiable tasks in browser games [5]; PhoneWorld turns phone-use trajectories into controllable environments and verification tasks [6]; ESI-Bench studies embodied spatial intelligence [1]; and ARC-AGI-3 studies interactive rule discovery in abstract environments [2]. AgentArk differs in scope: it is designed as a general-purpose multimodal task runtime rather than a benchmark tied to one environment family. The current suite spans 2D, 3D, physical, spatial, game-like, phone-use, and GUI-operation tasks, but the interface is intended to host other task forms that can be represented through multimodal observations, executable actions, and task-defined scoring.

Earlier embodied and agent-environment platforms provide important infrastructure for this setting. AI2-THOR and Habitat expose interactive 3D worlds for embodied perception and navigation [7, 8]; MineDojo scales open-ended embodied tasks in Minecraft [9]; and WebArena and AndroidWorld evaluate agents in web and mobile interfaces [10, 11]. AgentArk is complementary to these environments: its emphasis is not a single domain simulator, but a task-module interface that connects visual scenes, verifiable scoring, replay, code-as-action interaction, and RL-oriented execution.

For action representation, CodeAct shows that executable code can serve as a general action space for language-model agents [12], and Code as Policies demonstrates the feasibility of language models generating control policies as code [13]. AgentArk adopts this idea while constraining actions through the schema declared by each task, preserving flexibility while keeping the interaction verifiable and debuggable. More complex tasks can also allow a complete executable script to be submitted as a single action.

A related line studies environment and harness construction. GameCraft-Bench evaluates whether agents can build playable games end-to-end inside real game engines [14]. Agent-World and HarnessForge emphasize the co-evolution of agents, environments, and harnesses [15, 16], while OpenEnv attempts to standardize agentic RL environment interfaces [17]. AgentArk can be seen as one concrete multimodal execution-environment instance in this broader trend.

Table 1: Positioning of AgentArk with respect to adjacent research directions.

Research Direction	Representative Work	Primary Contribution	Relation to AgentArk
World models and general 3D agents	Genie 3, SIMA 2	Open, dynamic interactive worlds and agents that act in them	Complementary route: AgentArk emphasizes task goals, scoring, replay, and training interfaces, and can later use generated scenes or assets as richer variation sources.
Domain-specific interactive benchmarks	GameWorld, PhoneWorld, ESI-Bench, ARC-AGI-3	Verifiable tasks in games, phone use, embodied spatial reasoning, or rule discovery	AgentArk is a general-purpose task runtime: the current catalog spans several task families, while the interface is not tied to those categories.
Embodied, web, and mobile environments	AI2-THOR, Habitat, MineDojo, WebArena, AndroidWorld	Reusable simulators or interfaces for navigation, embodied control, web, and mobile agents	AgentArk adds a task-module layer that connects scene state, scoring, replay, code-as-action interaction, and RL execution across task families.
Code-as-action agents	CodeAct, Code as Policies	Executable code as a flexible action or control representation	AgentArk operationalizes code as action inside multimodal environments with schema validation, compilation feedback, runtime-error feedback, and replayable traces.
Environment generation and harness standards	GameCraft-Bench, Agent-World, HarnessForge, OpenEnv	Building or standardizing environments, harnesses, and agent-environment training loops	AgentArk is a concrete runtime and toolkit for scalable task creation, model evaluation, trace replay, and RL over loadable multimodal task modules.
AgentArk (this work)	This paper	General-purpose multimodal task modules with a reusable runtime shell, Python toolkit, Hub artifacts, replay, and RL interface	Designed as extensible infrastructure rather than a fixed benchmark; the public release demonstrates the framework with 32 listed tasks and 31 public scoreboards.

3 System Overview

AgentArk can be understood as a general-purpose trainable task platform. The environment runtime loads tasks, renders observations, and executes actions; task modules define concrete problems; the Python toolkit connects models, environments, evaluation runs, replay, and RL training; and the Hub site organizes task pages, results, and reproducibility artifacts. A task can combine image, video, or text observations, structured tool calls or executable code actions, and task-specific scoring rules. New multimodal tasks are therefore added as modules rather than by changing the core interface. The current runtime is implemented in Unity and uses Unity ML-Agents for agent-environment communication [18]. After this implementation detail, the paper refers to it simply as the runtime; the user-facing interface is the task instruction, observation, action, and score rather than the underlying implementation.

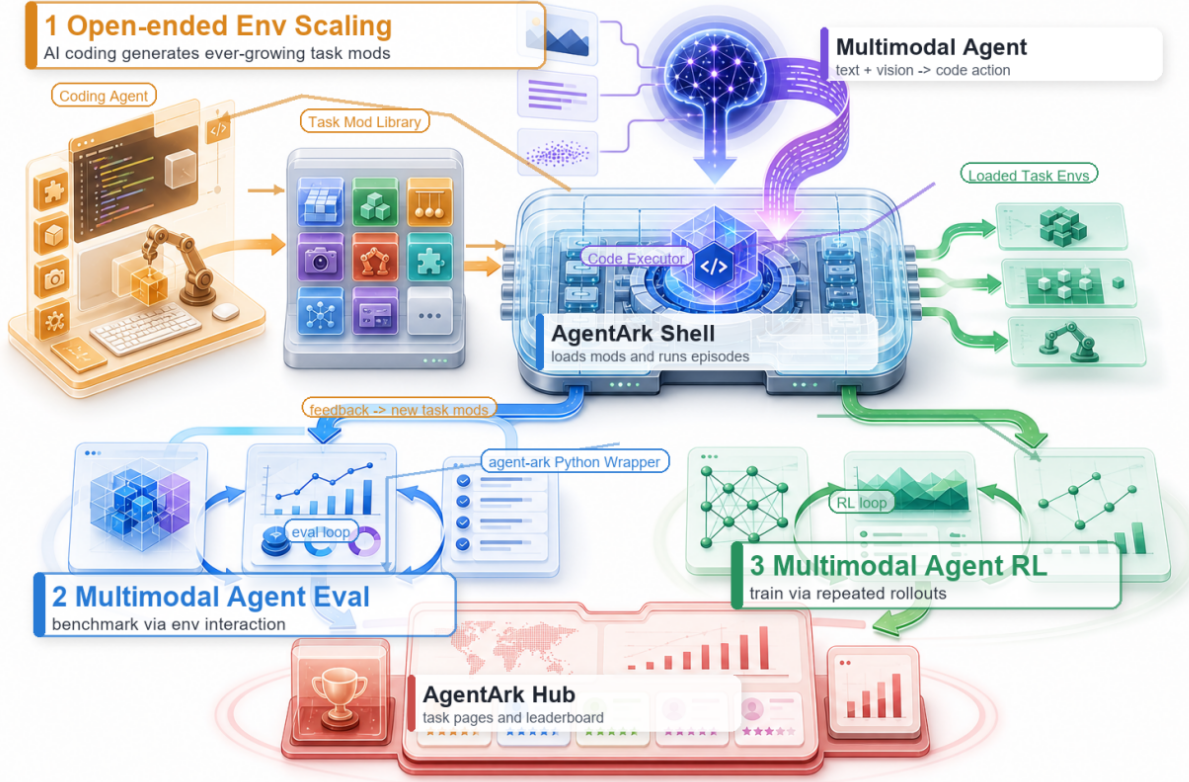


Figure 1: AgentArk overview. The system is organized around three loops: environment scaling, agent evaluation, and agent reinforcement learning.

Figure 1 shows three loops that also organize the rest of the paper. Environment scaling is the process of turning new task ideas into reusable task modules. Agent evaluation runs multimodal models against the same task definitions and stores traces for replay and analysis. Agent reinforcement learning reuses the same runtime and task modules through a service interface so that training and evaluation stay aligned. The runtime-shell design described below is the shared interaction layer used by these loops.

4 Environment Scaling: Multi-Agent Task Production

AgentArk scales environment supply through a coding-agent task production workflow rather than by manually building every environment from scratch. The current workflow is implemented with Codex and uses a coordinator to run preflight checks, route work across specialized subagents, and bound the repair loop. A designer agent turns a capability gap or task idea into a concrete task specification, a builder agent implements the scene, action interface, and scoring rule, and a reviewer agent checks visual clarity, solvability, replay readiness, and interaction evidence. When review finds issues, the coordinator routes focused fixes back to the builder for at most two fix-review rounds.

This workflow is how AgentArk task modules are produced and expanded over time. Released task modules can then be consumed directly by evaluation and RL jobs through the public runtime and Python toolkit. In current use, the automatic development cycle for an individual task typically takes about 0.5–1.5 hours, including design, implementation, visual checks, smoke tests, and replay-ready validation evidence.

Multi-agent task production workflow

A coordinator routes task design, implementation, review, and bounded fix-review rounds.

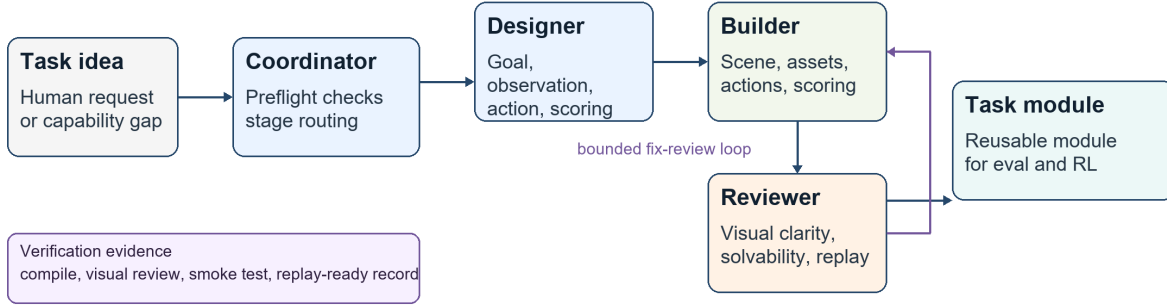


Figure 2: Environment scaling workflow. A coordinator routes specialized coding agents through design, implementation, review, bounded fix-review rounds, and verification evidence to produce reusable task modules.

A central lesson is that code correctness is not the same as task correctness. A multimodal task must also make its target visually observable, provide feedback sufficient for decision making, avoid scoring loopholes, and remain solvable across randomized instances. By moving these checks into task production, AgentArk can obtain stable task modules within short development cycles.

5 AgentArk Runtime Shell: Beyond Fixed Gym-Style Interfaces

Classic Gym-style environments standardize an important control loop: reset the environment, observe a fixed-shape state, choose an action from a fixed action space, and receive the next observation, reward, and termination flag [19, 20]. Before the rise of LLM-based agents, reinforcement-learning environments were commonly exposed through Gym-style interfaces with fixed observation and action spaces, because learned policies typically consumed and produced tensors of predefined shapes. Multimodal language-model agents have a different interface profile. They can read variable-length text, image sequences, tool documentation, error messages, and history; they can also produce structured text or executable programs rather than a single discrete or continuous vector. AgentArk keeps the same closed-loop spirit, but replaces fixed observation and action shapes with a task-declared, context-rich runtime shell.

5.1 Task Contract and Macro-Level Actions

Each task module exposes four kinds of information to the model: a natural-language instruction, image or text observations, a description of the allowed action format, and scoring and termination rules. The model does not need to directly simulate low-level keyboard, mouse, or controller actions; instead, it emits structured high-level actions or, for tasks that need more control, a complete executable script as the action. This design keeps evaluation focused on spatial reasoning, planning, parameter estimation, and tool composition.

This action abstraction is intentionally more macro-level than many conventional simulator actions. A single action may express a route plan, a sequence of GUI operations, a calibrated throw, or a small controller program that runs for a decision interval. The purpose is not to remove interaction, but to move the action space closer to the planning and tool-use level at which multimodal language agents naturally operate.

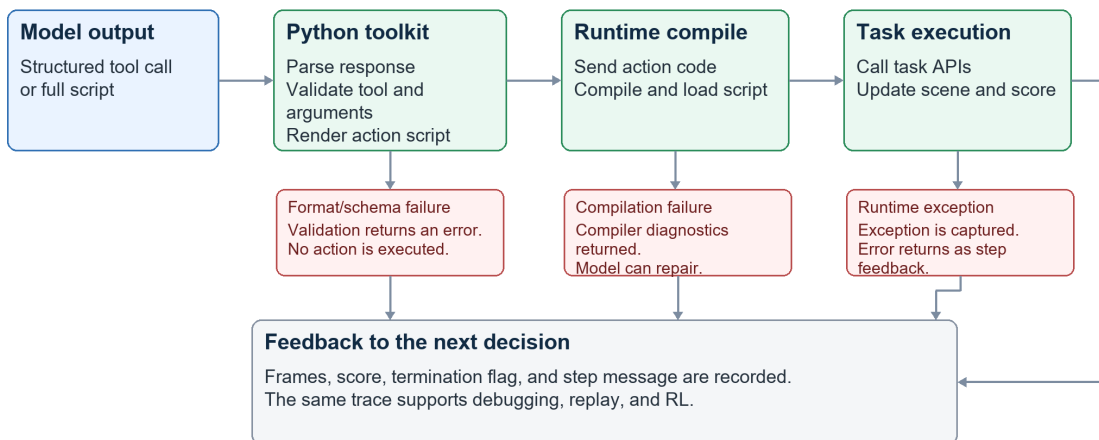
5.2 Action: Code as Action with Error Feedback

AgentArk adopts the general idea of code as action but adds an execution contract around it. In the common tool-call mode, a task exposes a machine-readable tool manifest. The model emits a structured tool call, the Python toolkit checks the JSON payload, tool name, required arguments, ranges, enumerations, and patterns, and then renders the

call into a small C# action script. In full-code mode, the model can submit a complete C# script directly, which is useful when one action must define a longer control procedure rather than a single tool call.

Code as action in AgentArk

Flexible model actions run as code; invalid formats, compile errors, and runtime exceptions become feedback.



Implementation evidence: validation, runtime compilation, exception feedback, and replayable traces.

Figure 3: AgentArk code-as-action execution path. Flexible model actions are converted into executable code, while invalid formats, compilation failures, and runtime exceptions are returned as explicit feedback.

Figure 3 highlights why the protocol is more than simply "let the model write code." Flexibility introduces new failure modes, so AgentArk turns them into observable environment feedback. First, Python-side validation catches malformed tool calls or schema violations before the runtime executes anything, and returns an explicit compile-stage message. Second, if the rendered or submitted script cannot be compiled by the runtime, compiler diagnostics are sent back to the model. Third, if the script compiles but fails during execution, the runtime captures the exception and returns it as the next step message. These messages are stored in the same trace as observations, scores, and termination flags, so errors can be used for repair, replay, and RL data.

5.3 Observation: Context Learning Across Steps and Attempts

The observation side follows the same principle. Instead of requiring a fixed observation tensor, AgentArk can return a structured multimodal context: current frames, task prompt, reset context, step messages, previous actions, previous scores, and selected history from earlier attempts. This is especially important for tasks whose rules, dynamics, or calibration parameters must be inferred through interaction. A failed attempt is not merely a discarded rollout; it can become evidence for the next decision.

Rollout, replay, and context learning

Failures and previous attempts can become context for later decisions.

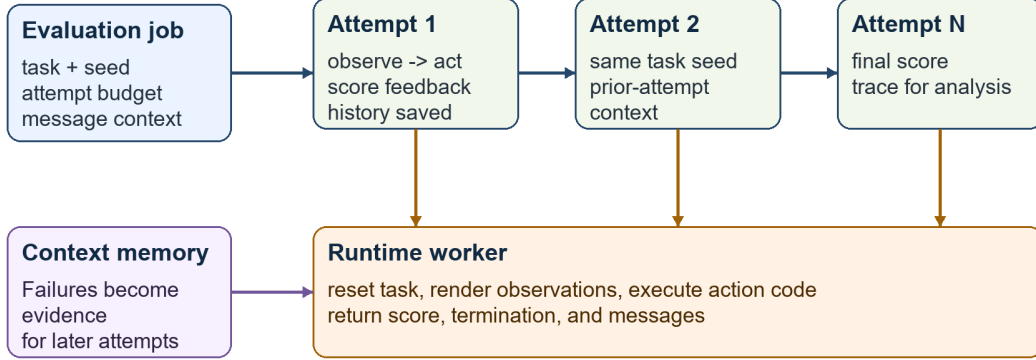


Figure 4: Contextual observation in AgentArk. A rollout can include multiple attempts; earlier failures become evidence for later decisions, and trace files support offline replay.

This context-learning design differs from training an agent to memorize one fixed environment. Task seeds, parameters, and layouts can vary, while the history mechanism exposes the agent’s own observations and mistakes. The goal is to evaluate whether a model can use environmental feedback as context, not merely whether it can execute a prelearned low-level policy.

5.4 Trace Semantics and Replay

Multimodal environments are harder to reproduce than text environments because rendering, physics, and random initialization can all introduce variation. AgentArk records the task, random seed, model actions, error messages, observations, scores, and termination information, and keeps random factors controlled by explicit seeds where possible. Replay does not call the model again; it re-executes recorded actions under the same task conditions and regenerates observations and scores. This supports model debugging, public reproducibility, and RL data inspection.

6 Agent Evaluation: Task Suite and Initial Results

The current public task catalog contains 32 listed tasks, with 31 public scoreboards in the Hub. It covers 2D and 3D scenes, static observations, short video observations, physical calibration, spatial relations, counting, path planning, mini-games, timing control, and GUI-operation tasks. This catalog demonstrates the range of the framework but is not meant to define its boundary: additional multimodal tasks can be introduced by adding task modules with their own observations, actions, and scoring rules. Each task was produced through the multi-agent task production workflow described above. Figures 5–7 show the public thumbnails and one-sentence objectives for catalog entries with released thumbnails.

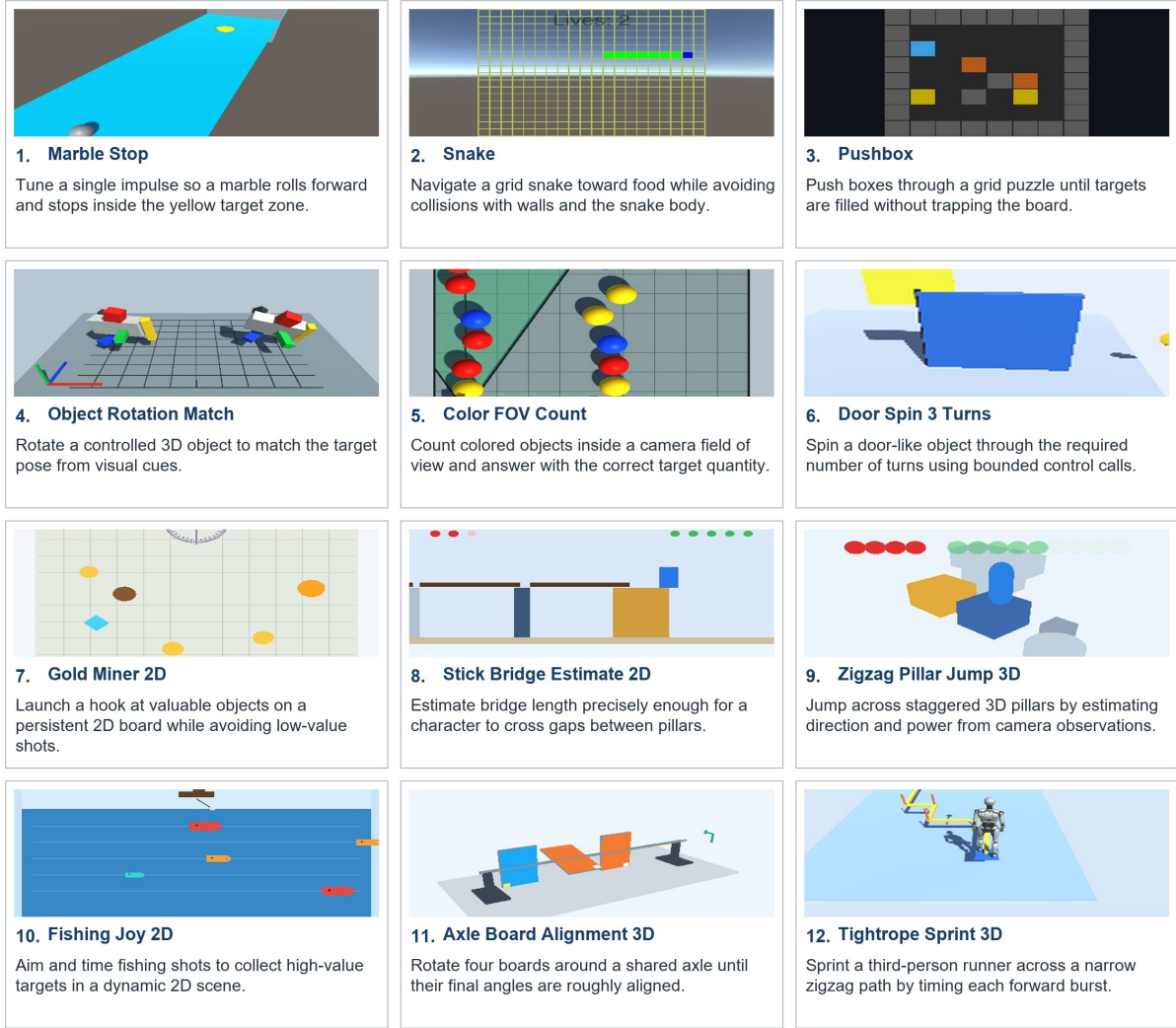


Figure 5: Thumbnails and one-sentence objectives for public thumbnail entries 1–12 in the AgentArk task catalog.

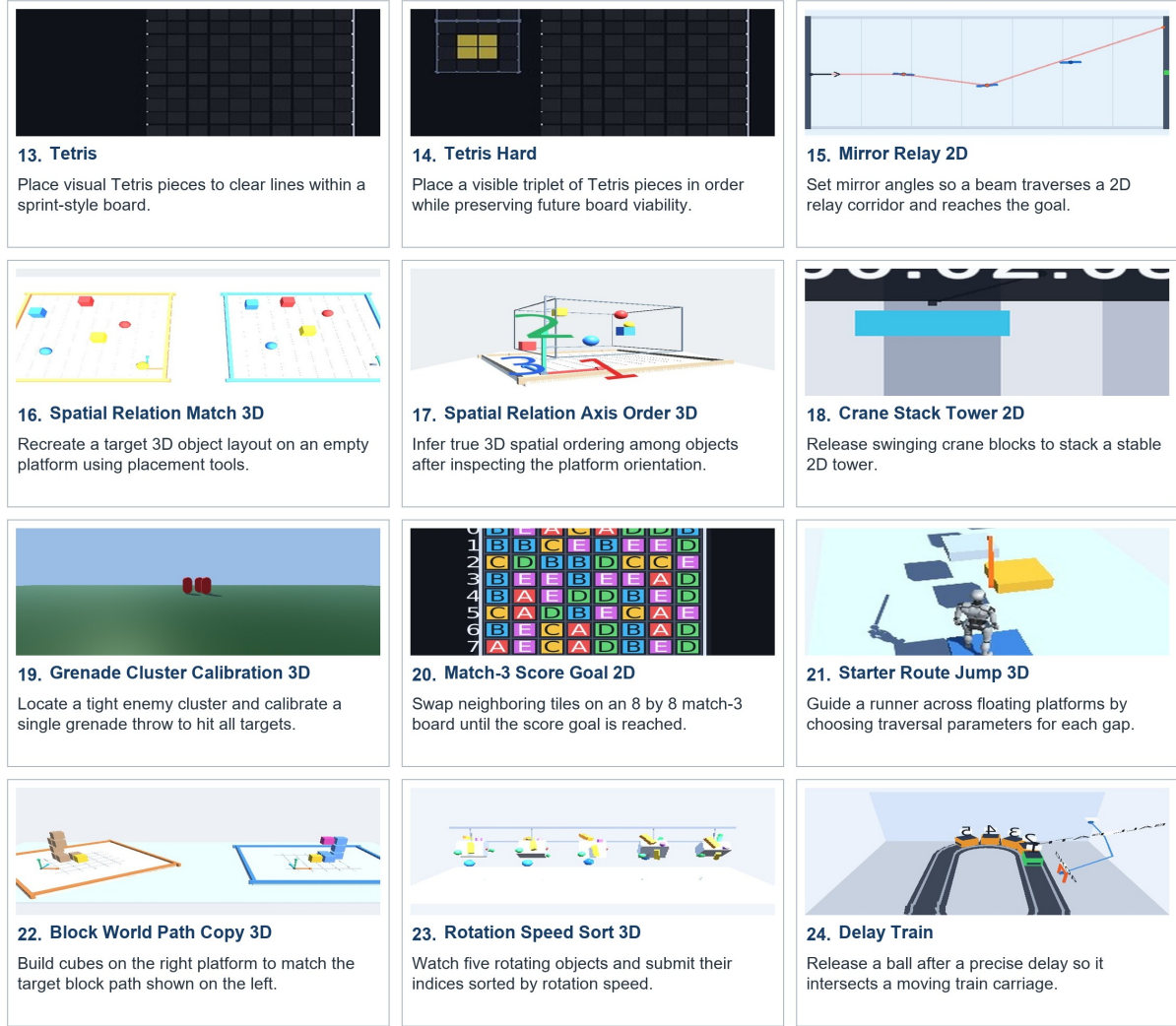


Figure 6: Thumbnails and one-sentence objectives for public thumbnail entries 13–24 in the AgentArk task catalog.

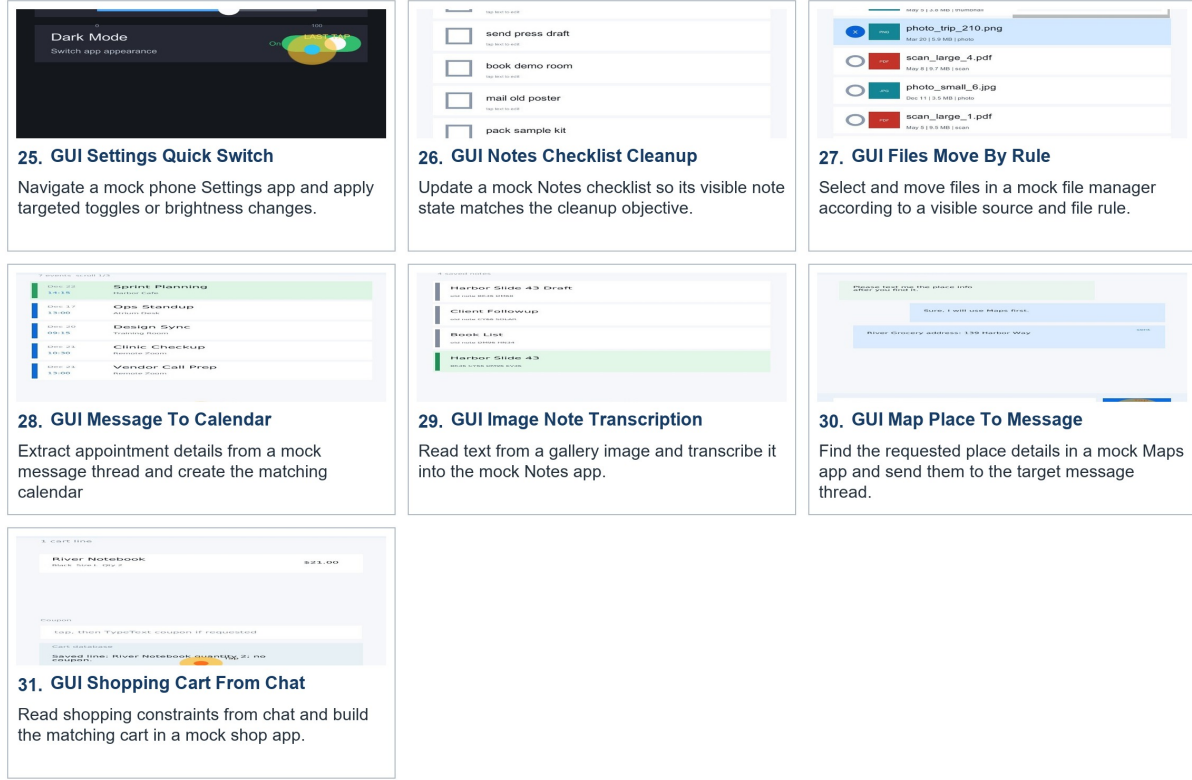


Figure 7: Thumbnails and one-sentence objectives for public thumbnail entries 25–31 in the AgentArk task catalog.

The current Hub results are computed from recorded model-environment traces over the 31 tasks with public scoreboards. All reported models were evaluated through the same task instructions, declared action schemas, scoring rules, and termination conditions. Table 2 compares automatic models that cover all 31 scoreboards. Champion counts follow the Hub convention: if several models tie for the best score on a task, each tied model receives a champion count. Therefore, the sum of champion counts can exceed the number of tasks. Scores are task-defined and aggregated by the Hub rather than normalized into an external benchmark scale. Because provider-hosted model endpoints can change over time, the model names should be read as labels for this public result snapshot rather than immutable model binaries. The release package is organized to include replay JSONL traces, task modules, runtime version metadata, and model configuration metadata for later reproduction.

Table 2: Automatic-model summary from AgentArk Hub. Champion counts include ties. The 3D and GUI champion columns are counted over 14 3D scoreboards and 7 GUI scoreboards, respectively.

Model	Tasks	Champions	Champ. Rate	3D Champs	GUI Champs	Avg. Success
gpt-5.5	31	22	71%	9	5	64%
gemini-3.5-flash	31	13	42%	5	6	51%
qwen37-plus	31	4	13%	3	1	28%
grok-4.20	31	2	6%	1	1	23%

In the current Hub results, gpt-5.5 remains strongest overall, with 22 champion counts across 31 scoreboards. The category split is more informative than a single global number. On the 14 3D scoreboards, gpt-5.5 records 9 champion counts and an average success rate of 69%, while gemini-3.5-flash records 5 champion counts and 49% average success. On the newer GUI-operation subset, gemini-3.5-flash is best or tied-best on 6 of 7 scoreboards, while gpt-5.5

is best or tied-best on 5. Several GUI tasks are saturated by full-score ties, so the GUI result should be read as an early capability signal rather than a final separation. The broader pattern is that gpt-5.5 is markedly stronger on 3D and non-GUI tasks, while gemini-3.5-flash is especially competitive on GUI-operation tasks. Because the Hub is a continuously updated presentation and reproducibility platform, these numbers should be read as an initial evaluation report rather than a frozen benchmark conclusion.

7 Agent Reinforcement Learning Interface

AgentArk uses the same tasks, actions, and scoring rules for evaluation and reinforcement learning. During evaluation, the Python toolkit maintains the interaction context and calls the model. During RL, the training framework can maintain its own trajectory history while the environment returns only the new observation and score for the current step. This separation keeps evaluation and training aligned while making the system easier to integrate with different training frameworks.

AgentArk RL serving architecture

RL code talks to a stable client API; the server owns workers, task selection, and task resources.

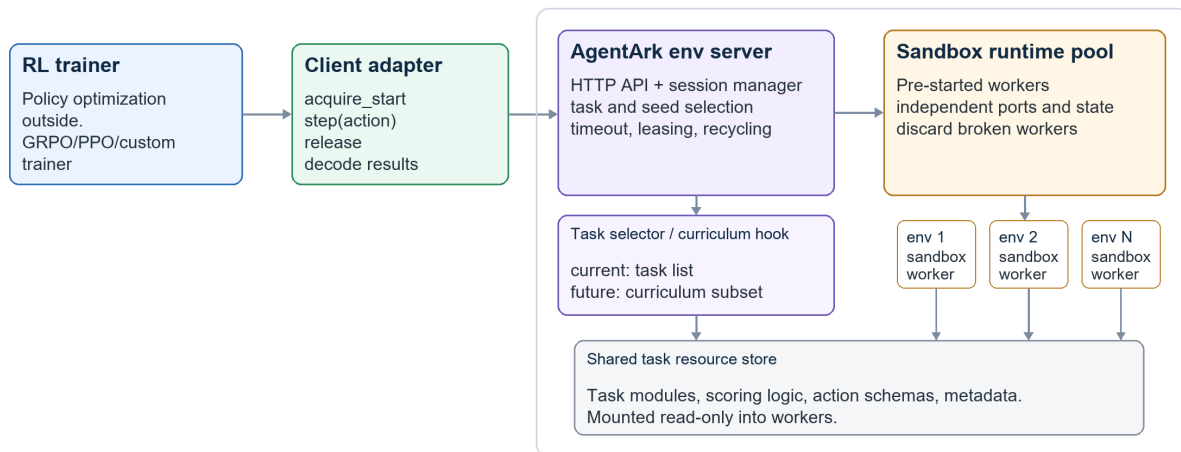


Figure 8: AgentArk RL serving architecture. Training frameworks use a client adapter, while the environment server manages runtime workers, sandbox copies, task resources, task selection, warmup, leasing, and recycling.

Figure 8 shows the serving design used for RL integration. The training framework talks to AgentArk through a client adapter and does not directly manage environment processes. The server owns environment lifecycle: it leases a worker, resolves a task and seed through a task selector, steps the action, returns the next observation, score, termination flag, and auxiliary information, and releases or recycles the worker when the interaction ends.

For parallel evaluation and RL, AgentArk runs isolated runtime workers while sharing a read-only task library. Each worker can use a sandboxed runtime copy with its own process state and port assignment, while task modules and metadata are shared as common resources. This prevents different tasks or model jobs from contaminating each other’s runtime state. A pool of pre-started environments can reduce cold-start overhead during long training runs, and broken or timed-out workers can be discarded rather than reused.

For group-comparison RL methods, AgentArk can ensure that samples in the same group use the same task and random seed, making differences between completions primarily reflect the policy. In the current implementation, the selector can draw from the available task list maintained by the environment manager. In future training setups, this hook can be connected to an evaluation or curriculum service that filters out tasks that are currently too easy or too hard and returns a task subset based on model success rates, error types, and capability gaps.

At the interface level, an RL trainer follows a simple acquire-start-step-release pattern. This keeps simulator lifecycle

management inside AgentArk while allowing external training code to own policy optimization and trajectory storage. Because the API boundary is HTTP-based, environment workers can run on separate machines or an environment cluster while training code runs elsewhere.

8 Discussion and Limitations

The main current limitation is not whether an individual task can be implemented quickly, but where task ideas come from. In the first release, humans primarily proposed capability gaps and task goals, while coding agents transformed them into runnable tasks. A natural next step is to let agents propose new tasks from failure traces and capability gaps.

A second limitation is context scale in long-horizon tasks. AgentArk intentionally allows failed attempts and environment feedback to enter the context of later attempts, which tests and trains context learning. However, very long tasks can cause traces to grow quickly. Future versions need local external memory that compresses observations, actions, scores, and error analyses into structured summaries.

9 Future Work

- Enable environment-model co-evolution: agents identify capability gaps, propose new tasks, implement task modules with their coding ability, train on those tasks, and then generate harder tasks.
- Scale the task store from the current public catalog to thousands of reproducible, trainable task modules with automatic checking mechanisms.
- Let agents propose task ideas from failure traces, capability gaps, and curriculum-learning needs.
- Develop local external memory for long-horizon tasks, compressing observations, actions, scores, and error analyses.
- Introduce dynamic curriculum learning based on model success rates, error types, task lists, and task parameters.
- Combine 3D generation and world models with AgentArk’s verifiable task logic.
- Strengthen the Hub with runtime and task-module versions, trace artifacts, download links, and more stable public model reports.

10 Availability

The AgentArk project is publicly available at <https://github.com/P90-RushB/AgentArk> [21]. The live AgentArk Hub is available at <https://p90-rushb.github.io/agentark-hub/> [22]. The Hub provides task pages, public leaderboards, task artifacts, replay traces, and metadata for reproducing reported interactions.

11 Conclusion

AgentArk aims to move multimodal-agent environments from static task sets toward continuously extensible training infrastructure. Task modules define concrete problems, the runtime provides visible interactive worlds, the Python toolkit unifies evaluation, replay, parallel execution, and RL, and the Hub site organizes tasks and results. The current public catalog and Hub results show the feasibility of this direction: models can not only learn inside environments, but also participate in proposing and constructing the next generation of training environments, forming a long-term co-evolution loop between agents and their training worlds.

References

- [1] Yining Hong et al. ESI-Bench: Towards embodied spatial intelligence that closes the perception-action loop, 2026. arXiv:2605.18746.
- [2] ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence, 2026. arXiv:2603.24621.
- [3] Google DeepMind. Genie 3: A new frontier for world models. <https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/>, 2026.
- [4] SIMA Team, Adrian Bolton, et al. SIMA 2: A generalist embodied agent for virtual worlds, 2026. arXiv:2512.04797.
- [5] Mingyu Ouyang, Siyuan Hu, Kevin Qinghong Lin, Hwee Tou Ng, and Mike Zheng Shou. GameWorld: Towards standardized and verifiable evaluation of multimodal game agents, 2026. arXiv:2604.07429.
- [6] Zhengyang Tang et al. PhoneWorld: Scaling phone-use agent environments, 2026. arXiv:2605.29486.
- [7] Eric Kolve, Roozbeh Mottaghi, Winson Han, et al. AI2-THOR: An interactive 3D environment for visual AI, 2017. arXiv:1712.05474.
- [8] Manolis Savva, Abhishek Kadian, Oleksandr Maksymets, et al. Habitat: A platform for embodied AI research, 2019. arXiv:1904.01201.
- [9] Linxi Fan, Guanzhi Wang, Yunfan Jiang, et al. MineDojo: Building open-ended embodied agents with internet-scale knowledge, 2022. arXiv:2206.08853.
- [10] Shuyan Zhou, Frank F. Xu, Hao Zhu, et al. WebArena: A realistic web environment for building autonomous agents, 2023. arXiv:2307.13854.
- [11] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, et al. AndroidWorld: A dynamic benchmarking environment for autonomous agents, 2024. arXiv:2405.14573.
- [12] Xingyao Wang et al. Executable code actions elicit better LLM agents, 2024. arXiv:2402.01030.
- [13] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as Policies: Language model programs for embodied control, 2022. arXiv:2209.07753.
- [14] Tongxu Luo et al. GameCraft-Bench: Can agents build playable games end-to-end in a real game engine?, 2026. arXiv:2606.17861.
- [15] Guanting Dong et al. Agent-World: Scaling real-world environment synthesis for evolving general agent intelligence, 2026. arXiv:2604.18292.
- [16] Mingju Chen, Can Lv, Guibin Zhang, Heng Chang, and Shiji Zhou. HarnessForge: Joint harness and policy evolution for adaptive agent systems, 2026. arXiv:2606.01779.
- [17] Hugging Face. OpenEnv: Agentic execution environments. <https://github.com/huggingface/OpenEnv>, 2026.
- [18] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents, 2018. arXiv:1809.02627.
- [19] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016. arXiv:1606.01540.
- [20] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024. arXiv:2407.17032.

- [21] Jiage Liang. AgentArk: A general-purpose environment framework for multimodal agents. <https://github.com/P90-RushB/AgentArk>, 2026.
- [22] Jiage Liang. AgentArk Hub. <https://p90-rushb.github.io/agentark-hub/>, 2026.